

Assorted NB comment resolutions for Kona 2025

Document #: P3914R0
Date: 2025-11-04
Project: Programming Language C++
Audience: LWG
Reply-to: Tim Song
<t.canens.cpp@gmail.com>

Contents

[1 Introduction](#)

[2 Wording](#)

[2.1 US 160-260](#)

[2.2 US 209-332](#)

[2.3 US 228-348](#)

[2.4 US 263-396](#)

[2.5 US 265-398 and US 266-399](#)

[2.6 US 112-172](#)

[2.7 US 130-193](#)

[3 References](#)

§ 1 Introduction

This paper provides wording to resolve the following national body comments on the C++26 CD:

- [US 160-260](#)
- [US 209-332](#)
- [US 228-348](#)
- [US 263-396](#)
- [US 265-398](#)
- [US 266-399](#)
- [US 112-172](#)
- [US 130-193](#)

§ 2 Wording

This wording is relative to [\[N5014\]](#) except where noted.

§ 2.1 US 160-260

[Drafting note: During offline discussion, Ruslan Arutyunyan pointed out other issues with the specification of `unique` and `unique_copy` and suggested an elegant wording strategy to address them. The wording below incorporates his suggestions.]

Edit 26.7.9 [\[alg.unique\]](#) as indicated:

```
template<class ForwardIterator>
    constexpr ForwardIterator unique(ForwardIterator first, ForwardIterator last);
template<class ExecutionPolicy, class ForwardIterator>
    ForwardIterator unique(ExecutionPolicy&& exec,
                          ForwardIterator first, ForwardIterator last);

template<class ForwardIterator, class BinaryPredicate>
    constexpr ForwardIterator unique(ForwardIterator first, ForwardIterator last,
                                     BinaryPredicate pred);
template<class ExecutionPolicy, class ForwardIterator, class BinaryPredicate>
    ForwardIterator unique(ExecutionPolicy&& exec,
                          ForwardIterator first, ForwardIterator last,
                          BinaryPredicate pred);

template<permutable I, sentinel_for<I> S, class Proj = identity,
        indirect_equivalence_relation<projected<I, Proj>> C = ranges::equal_to>
    constexpr subrange<I> ranges::unique(I first, S last, C comp = {}, Proj proj = {});
template<forward_range R, class Proj = identity,
        indirect_equivalence_relation<projected<iterator_t<R>, Proj>> C =
        ranges::equal_to>
    requires permutable<iterator_t<R>>
    constexpr borrowed_subrange_t<R>
        ranges::unique(R&& r, C comp = {}, Proj proj = {});

template<execution_policy Ep, random_access_iterator I, sized_sentinel_for<I> S,
        class Proj = identity,
        indirect_equivalence_relation<projected<I, Proj>> C = ranges::equal_to>
    requires permutable<I>
    subrange<I> ranges::unique(Ep&& exec, I first, S last, C comp = {}, Proj proj = {});
template<execution_policy Ep, sized_random_access_range R, class Proj = identity,
        indirect_equivalence_relation<projected<iterator_t<R>, Proj>> C =
        ranges::equal_to>
    requires permutable<iterator_t<R>>
    borrowed_subrange_t<R> ranges::unique(Ep&& exec, R&& r, C comp = {}, Proj proj =
        {});
```

- 1 Let `pred` be `equal_to{}` for the overloads with no parameter `pred`, and let $E(i)$ be
 - (1.?) — `false` if i is equal to `first`; otherwise
 - (1.1) — `bool(pred(*(i - 1), *i))` for the overloads in namespace `std`;
 - (1.2) — `bool(invoker(comp, invoker(proj, *(i - 1)), invoker(proj, *i)))` for the overloads in namespace `ranges`.
- 2 *Preconditions:* For the overloads in namespace `std`, `pred` is an equivalence relation and the type of `*first` meets the *Cpp17MoveAssignable* requirements (Table 33)

[\[tab:cpp17.moveassignable\]](#))).

- 3 *Effects:* For a nonempty range, eliminates all but the first element from every consecutive group of equivalent elements referred to by the iterator `i` in the range `[first + 1, last)` for which `E(i)` is `true`.
- 4 *Returns:* Let `j` be the end of the resulting range. Returns:
 - (4.1) — `j` for the overloads in namespace `std`.
 - (4.2) — `{j, last}` for the overloads in namespace `ranges`.
- 5 *Complexity:* For nonempty ranges, exactly $(last - first) - 1$ applications of the corresponding predicate and no more than twice as many applications of any projection.

```
template<class InputIterator, class OutputIterator>
constexpr OutputIterator
    unique_copy(InputIterator first, InputIterator last,
                OutputIterator result);
template<class ExecutionPolicy, class ForwardIterator1, class ForwardIterator2>
ForwardIterator2
    unique_copy(ExecutionPolicy&& exec,
                ForwardIterator1 first, ForwardIterator1 last,
                ForwardIterator2 result);

template<class InputIterator, class OutputIterator,
         class BinaryPredicate>
constexpr OutputIterator
    unique_copy(InputIterator first, InputIterator last,
                OutputIterator result, BinaryPredicate pred);
template<class ExecutionPolicy, class ForwardIterator1, class ForwardIterator2,
         class BinaryPredicate>
ForwardIterator2
    unique_copy(ExecutionPolicy&& exec,
                ForwardIterator1 first, ForwardIterator1 last,
                ForwardIterator2 result, BinaryPredicate pred);

template<input_iterator I, sentinel_for<I> S, weakly_incrementable O, class Proj =
    identity,
        indirect_equivalence_relation<projected<I, Proj>> C = ranges::equal_to>
requires indirectly_copyable<I, O> &&
    (forward_iterator<I> ||
     (input_iterator<O> && same_as<iter_value_t<I>, iter_value_t<O>>) ||
     indirectly_copyable_storable<I, O>)
constexpr ranges::unique_copy_result<I, O>
    ranges::unique_copy(I first, S last, O result, C comp = {}, Proj proj = {});
template<input_range R, weakly_incrementable O, class Proj = identity,
        indirect_equivalence_relation<projected<iterator_t<R>, Proj>> C =
    ranges::equal_to>
requires indirectly_copyable<iterator_t<R>, O> &&
    (forward_iterator<iterator_t<R>> ||
     (input_iterator<O> && same_as<range_value_t<R>, iter_value_t<O>>) ||
     indirectly_copyable_storable<iterator_t<R>, O>)
constexpr ranges::unique_copy_result<borrowed_iterator_t<R>, O>
    ranges::unique_copy(R&& r, O result, C comp = {}, Proj proj = {});
```

```

template<execution-policy Ep, random_access_iterator I, sized_sentinel_for<I> S,
        random_access_iterator O, sized_sentinel_for<O> OutS, class Proj = identity,
        indirect_equivalence_relation<projected<I, Proj>> C = ranges::equal_to>
requires indirectly_copyable<I, O>
ranges::unique_copy_result<I, O>
ranges::unique_copy(Ep&& exec, I first, S last, O result, OutS result_last,
                    C comp = {}, Proj proj = {});
template<execution-policy Ep, sized-random-access-range R, sized-random-access-range
        OutR,
        class Proj = identity,
        indirect_equivalence_relation<projected<iterator_t<R>, Proj>> C =
        ranges::equal_to>
requires indirectly_copyable<iterator_t<R>, iterator_t<OutR>>
ranges::unique_copy_result<borrowed_iterator_t<R>, borrowed_iterator_t<OutR>>
ranges::unique_copy(Ep&& exec, R&& r, OutR&& result_r, C comp = {}, Proj proj =
                    {});

```

6 Let `pred` be `equal_to{}` for the overloads in namespace `std` with no parameter `pred`, and let $E(i)$ be

- (6.?) — `false` if i is equal to `first`; otherwise
- (6.1) — `bool(pred(*i, *(i - 1)))` for the overloads in namespace `std`;
- (6.2) — `bool(invoke(comp, invoke(proj, *i), invoke(proj, *(i - 1))))` for the overloads in namespace `ranges`.

7 Let:

- (7.1) — M be the number of iterators i in the range `[first + 1, last)` for which $E(i)$ is `false`;
- (7.2) — `result_last` be `result + M + 1` for the overloads with no parameter `result_last` or `result_r`;
- (7.3) — N be `min(M + 1, result_last - result)`

8 *Mandates:* `*first` is writable (24.3.1 [\[iterator.requirements.general\]](#)) to `result`.

9 *Preconditions:*

- (9.1) — The ranges `[first, last)` and `[result, result + N)` do not overlap.
- (9.2) — For the overloads in namespace `std`:
 - (9.2.1) — The comparison function is an equivalence relation.
 - (9.2.2) — For the overloads with no `ExecutionPolicy`, let T be the value type of `InputIterator`. If `InputIterator` models `forward_iterator` (24.3.4.11 [\[iterator.concept.forward\]](#)), then there are no additional requirements for T . Otherwise, if `OutputIterator` meets the *Cpp17ForwardIterator* requirements and its value type is the same as T , then T meets the *Cpp17CopyAssignable* (Table 34 [\[tab.cpp17.copyassignable\]](#)) requirements. Otherwise, T meets both the

Cpp17CopyConstructible (Table 32 [\[tab:cpp17.copyconstructible\]](#)) and *Cpp17CopyAssignable* requirements.

[*Note 1*: For the parallel algorithm overloads in namespace `std`, there can be a performance cost if the value type of `ForwardIterator1` does not meet both the *Cpp17CopyConstructible* and *Cpp17CopyAssignable* requirements. For the parallel algorithm overloads in namespace `ranges`, there can be a performance cost if `iter_value_t<I>` does not model copyable.
— *end note*]

10 *Effects*: Copies only the first N element from N consecutive groups of equivalent elements referred to by the iterator `i` in the range `[first + 1, last)` for which $E(i)$ holds is false into the range `[result, result + N)`.

11 *Returns*:

(11.1) — `result + N` for the overloads in namespace `std`.

(11.2) — `{last, result + N}` for the overloads in namespace `ranges`, if N is equal to $M + 1$.

(11.3) — Otherwise, `{j, result_last}` for the overloads in namespace `ranges`, where `j` is the iterator in `[first + 1, last)` for which $E(j)$ is false and there are exactly $N - 1$ iterators `i` in `[first + 1, j)` for which $E(i)$ is false.

12 *Complexity*: At most `last - first - 1` applications of the corresponding predicate and no more than twice as many applications of any projection.

§ 2.2 [US 209-332](#)

1. Add the following to the end of 33.9.1 [\[exec.snd.general\]](#):

? Various function templates in subclause 33.9 [\[exec.snd\]](#) can throw an exception of type *unspecified-exception*. Each such exception object is of an unspecified type such that a *handler* of type `exception` matches (14.4 [\[except.handle\]](#)) the exception object but a *handler* of type `dependent_sender_error` does not. [*Note 1*: There is no requirement that two such exception objects have the same type. — *end note*]

2. Edit 33.9.2 [\[exec.snd.expos\]](#) as indicated:

```
47 struct not-a-sender {
    using sender_concept = sender_t;

    template<class Sndr>
        static constexpr auto get_completion_signatures() ->
            completion_signatures<> {
            throw unspecified-exception();
        }
};
```

where *unspecified-exception* is a type derived from `exception`.

```
constexpr void decay-copyable-result-datums(auto cs) {
    cs.for-each([]<class Tag, class... Ts>(Tag(*) (Ts...)) {
        if constexpr (!is_constructible_v<decay_t<Ts>, Ts> &&...))
            throw unspecified-exception();
    });
}
```

where *unspecified-exception* is a type derived from *exception*.

3. Edit 33.9.11.3 [\[exec.read.env\]](#) as indicated:

```
template<class Sndr, class Env>
    static consteval void check-types();

4    Let Q be decay_t<data-type<Sndr>>.

5    Throws: An exception of type unspecified-exception (33.9.1
\[exec.snd.general\]) an unspecified type derived from exception if the
expression Q()(env) is ill-formed or has type void, where env is an lvalue
subexpression whose type is Env.
```

4. Edit 33.9.12.9 [\[exec.then\]](#) as indicated:

```
template<class Sndr, class... Env>
    static consteval void check-types();

5    Effects: Equivalent to:

        auto cs = get_completion_signatures<child-type<Sndr>,
            FWD-ENV-T(Env)...>();
        auto fn = []<class... Ts>(set_value_t(*) (Ts...)) {
            if constexpr (!invocable<remove_cvref_t<data-
                type<Sndr>>, Ts...>)
                throw unspecified-exception();
        };
        cs.for-each(overload-set{fn, [](auto){}});

    where unspecified-exception is a type derived from exception.
```

5. Edit 33.9.12.10 [\[exec.let\]](#) as indicated:

```
template<class Sndr, class... Env>
    static consteval void check-types();

7    Effects: Equivalent to:

        using LetFn = remove_cvref_t<data-type<Sndr>>;
        auto cs = get_completion_signatures<child-type<Sndr>,
            FWD-ENV-T(Env)...>();
        auto fn = []<class... Ts>(decayed-typeof<set-cpo>(*)
            (Ts...)) {
            if constexpr (!is-valid-let-sender)    // see below
                throw unspecified-exception();
        };
```

```
};
cs.for-each(overload-set(fn, [] (auto){}));
```

where *unspecified-exception* is a type derived from `exception`, and where *is-valid-let-sender* is `true` if and only if all of the following are `true`:

- (7.1) — `(constructible_from<decay_t<Ts>, Ts> && ...)`
- (7.2) — `invocable<LetFn, decay_t<Ts>&...>`
- (7.3) — `sender<invoke_result_t<LetFn, decay_t<Ts>&...>>`
- (7.4) — `sizeof...(Env) == 0 || sender_in<invoke_result_t<LetFn, decay_t<Ts>&...>, env-t...>`

where *env-t* is the pack `decltype(let-cpo.transform_env(declval<Sndr>(), declval<Env>()))`.

6. Edit 33.9.12.11 [\[exec.bulk\]](#) as indicated:

```
template<class Sndr, class... Env>
static constexpr void check-types();
```

7 *Effects*: Equivalent to:

```
auto cs = get_completion_signatures<child-type<Sndr>,
FWD-ENV-T(Env)...>();
auto fn = []<class... Ts>(set_value_t*)(Ts...) {
    if constexpr (!invocable<remove_cvref_t<data-
type<Sndr>>, Ts&...>)
        throw unspecified-exception();
};
cs.for-each(overload-set(fn, [] (auto){}));
```

where *unspecified-exception* is a type derived from `exception`.

7. Edit 33.9.12.12 [\[exec.when.all\]](#) as indicated:

```
template<class Sndr, class... Env>
static constexpr void check-types();
```

7 Let *Is* be the pack of integral template arguments of the `integer_sequence` specialization denoted by *indices-for<Sndr>*.

8 *Effects*: Equivalent to:

```
auto fn = []<class Child>() {
    auto cs = get_completion_signatures<Child, when-all-
env<Env>...>();
    if constexpr (cs.count-of(set_value) >= 2)
        throw unspecified-exception();
    decay-copyable-result-datums(cs); // see
[exec.snd.expos]
};
(fn.template operator()<child-type<Sndr, Is>>(), ...);
```

where *unspecified-exception* is a type derived from *exception*.

- 9 Throws: Any exception thrown as a result of evaluating the *Effects*, or an exception of type *unspecified-exception* (33.9.1 [exec.snd.general]) and unspecified type derived from *exception* when CD is ill-formed.

8. Edit 33.9.12.14 [exec.stopped.opt] as indicated:

- 3 The exposition-only class template *impls-for* (33.9.2 [exec.snd.expos]) is specialized for *stopped_as_optional_t* as follows:

```
namespace std::execution {
    template<>
    struct impls-for<stopped_as_optional_t> : default-impls {
        template<class Sndr, class... Env>
        static consteval void check-types() {
            default-impls::check-types<Sndr, Env...>();
            if constexpr (!requires {
                requires (!same_as<void, single-sender-value-type<child-
                    type<Sndr>,
                                                                    FWD-ENV-
                    T(Env)...>>); })
                throw unspecified-exception();
        }
    };
}
```

where *unspecified-exception* is a type derived from *exception*.

§ 2.3 [US 228-348](#)

[Drafting note: The font of *ssource-t* is corrected by [PR 8404](#).]

Edit 33.9.12.18 [exec.spawn.future] as indicated:

- 6 Let *ssource-t* be an unspecified type that models *stoppable-source* and *default_initializable*, such that a default-initialized object of type *ssource-t* has an associated stop state. Let *let* *ssource* be an lvalue of type *ssource-t*. Let *stoken-t* be `decltype(ssource.get_token())`. Let *future-spawned-sender* be the alias template:

```
template<sender Sender, class Env>
using future-spawned-sender =                                // exposition only
    decltype(write_env(stop-when(declval<Sender>(), declval<stoken-t>()), declval<Env>
        ()));
```

§ 2.4 [US 263-396](#)

[Drafting note: LWG wishes to allow an implementation that returns an object of static storage duration under appropriate conditions.]

Edit 33.16.3 [exec.sysctxrepl.query] as indicated:


```
shared_ptr<parallel_scheduler_backend> query_parallel_scheduler_backend();
```

- 1 *query_parallel_scheduler_backend()* returns the implementation object for a parallel scheduler.
- 2 *Returns:* A non-null shared pointer. An object `p`, such that `p.get()` points to a `parallel_scheduler_backend` object that is a base-class subobject of some most derived object `o` within its lifetime. The lifetime of `o` does not end as long as there exists a `shared_ptr` object `q` within its lifetime such that `q.owner_equal(p)` is `true`. to an object that implements the `parallel_scheduler_backend` interface.
- 3 *Remarks:* This function is replaceable (9.6.5 [\[dcl.fct.def.replace\]](#)).

§ 2.5 US 265-398 and US 266-399

[Editor's note: This wording is relative to the current draft, which already includes a [change](#) intended to address [US 266-399](#).]

Edit 33.16.4 [\[exec.sysctxrepl.psb\]](#) as indicated:

```
virtual void schedule(receiver_proxy& r, span<byte> s) noexcept = 0;
```

- 1 *Preconditions:* The ends of the lifetimes of `*this`, of the lifetime of the object referred to by `r`, and of the duration of any storage referenced by `s` all happen after the beginning of the evaluation of the call to `set_value`, `set_error`, or `set_done` `set_stopped` on `r` (see below).
- 2 *Effects:* [...]
- 3 *Remarks:* [...]

```
virtual void schedule_bulk_chunked(size_t n, bulk_item_receiver_proxy& r,  
                                  span<byte> s) noexcept = 0;
```

- 4 *Preconditions:* The ends of the lifetimes of `*this`, of the lifetime of the object referred to by `r`, and of the duration of any storage referenced by `s` all happen after the beginning of the evaluation of the call to `set_value`, `set_error`, or `set_stopped` on `r` (see below) one of the expressions below.
- 5 *Effects:* [...]
- 6 *Remarks:* [...]

```
virtual void schedule_bulk_unchunked(size_t n, bulk_item_receiver_proxy& r,  
                                     span<byte> s) noexcept = 0;
```

- 7 *Preconditions:* The ends of the lifetimes of `*this`, of the lifetime of the object referred to by `r`, and of the duration of any storage referenced by `s` all happen after the beginning of the evaluation of the call to `set_value`, `set_error`, or `set_stopped` on `r` (see below) one of the expressions below.

8 *Effects:* [...]

9 *Remarks:* [...]

§ 2.6 US 112-172

Edit 21.4.13 [[meta.reflection.extract](#)] as indicated:

```
template<class T>
constexpr T extract-value(info r);    // exposition only
```

8 Let *U* be the type of the value or object that *r* represents.

9 *Returns:* `static_cast<T>([:R:])`, where *R* is a constant expression of type *info* such that *R == r* is `true`.

10 *Throws:* `meta::exception` unless

- (10.1) — *U* is a pointer type, *T* and *U* are either similar (7.3.6 [[conv.qual](#)]) or both function pointer types, and `is_convertible_v<U, T>` is `true`,
- (10.2) — *U* is not a pointer type and the cv-unqualified types of *T* and *U* are the same,
- (10.3) — *U* is an array type, *T* is a pointer type, `remove_extent_t<U>*` and *T* are similar types, and the value *r* represents is convertible to *T*, or
- (10.4) — *U* is a closure type, *T* is a function pointer type, and the value that *r* represents is convertible to *T*.

§ 2.7 US 130-193

[Drafting note: There are a few cases to consider here.

1. A *Mandates:* clause was violated (e.g., `make_signed(^^float)`)
2. A member type alias type does not exist (e.g., `invoke_result(^^int, {})`)
3. A member value does not exist, or has the wrong type, or is inaccessible, etc. (`tuple_size(^^int)`)
4. [The general rule against incomplete types](#) is violated.
5. A precondition of a type trait is violated.
6. Evaluation of the type trait would trigger a hard error outside the trait.

For cases 1-3, we require an exception to be thrown.

For case 4, this wording requires a hard error (non-constant); this leaves open the possibility of evolving in the future to either relax the precondition (because functions can be called many times without ODR concerns) or enforce it via throw.

For case 5, some preconditions of type traits can be overly strict. Once case 4 is applied, the remaining cases can in theory produce an answer. The wording makes it unspecified whether the call is non-constant or produces the expected result.

Case 6 remains a hard error; we do not explicitly call this out since it's just like any template instantiation triggered by a call to a reflection function.]

1. Add the following paragraph to 21.4.17 [\[meta.reflection.traits\]](#) after p2:

- ? For a function or function template F defined in this subclause, let C be its associated class template. For the evaluation of a call to F , let S be the specialization of C in terms of which the call is specified.

(?.1) — If

(?.1.1) — the template arguments of S violate a condition specified in a *Mandates* element in the specification of C ;

(?.1.2) — the call is specified to produce a reflection of a type, but S would have no member named `type`; or

(?.1.3) — the call is specified to return `S::value`, but that expression is not a valid converted constant expression of type R , where R is the return type of F ;

then an exception of type `meta::exception` is thrown. [*Note 1*: For the first case, S is not instantiated. — *end note*]

(?.2) — Otherwise, if the instantiation of S would result in undefined behavior due to dependence on an incomplete type (21.3.2 [\[meta.rqmts\]](#)), then the call is not a constant subexpression.

(?.3) — Otherwise, if the template arguments of S do not meet the preconditions of C , then it is unspecified whether the call is a constant subexpression. If it is, the call produces the result that would be produced if C had no preconditions.

2. Edit 21.4.17 [\[meta.reflection.traits\]](#) as indicated:

Table 64 [tab:meta.reflection.traits] — Reflection type traits	
Signature and Return Type	Returns
<code>bool meta::UNARY(info type);</code> <code>bool meta::UNARY_type(info type);</code>	<code>std::UNARY_v<T>::value</code> , where T is the type or type alias represented by type
<code>bool meta::BINARY(info t1, info t2);</code> <code>bool meta::BINARY_type(info t1, info t2);</code>	<code>std::BINARY_v<T₁, T₂>::value</code> , where T_1 and T_2 are the types or type aliases represented by <code>t1</code> and <code>t2</code> , respectively

Signature and Return Type	Returns
<pre>template <reflection_range R> bool meta::VARIADIC_type(info type, R&& args);</pre>	<code>std::VARIADIC_v<T, U...>::value</code> where T is the type or type alias represented by type and $U...$ is the pack of types or type aliases whose elements are represented by the corresponding elements of args
<pre>template <reflection_range R> bool meta::VARIADIC_type(info t1, info t2, R&& args);</pre>	<code>std::VARIADIC_v<T₁, T₂, U...>::value</code> where T_1 and T_2 are the types or type aliases represented by t1 and t2, respectively, and $U...$ is the pack of types or type aliases whose elements are represented by the corresponding elements of args
<pre>info meta::UNARY(info type);</pre>	A reflection representing the type denoted by <code>std::UNARY_t<T>::type</code>, where T is the type or type alias represented by type
<pre>template <reflection_range R> info meta::VARIADIC(R&& args);</pre>	A reflection representing the type denoted by <code>std::VARIADIC_t<T...>::type</code> where $T...$ is the pack of types or type aliases whose elements are represented by the corresponding elements of args
<pre>template <reflection_range R> info meta::VARIADIC(info type, R&& args);</pre>	A reflection representing the type denoted by <code>std::VARIADIC_t<T, U...>::type</code> where T is the type or type alias represented by type and $U...$ is the pack of types or type aliases whose elements are represented by the corresponding elements of args

```
constexpr size_t rank(info type);
```

- ⁶ Returns: `rank_v<T>std::rank<T>::value`, where T is the type represented by `dealias(type)`.

```
constexpr size_t extent(info type, unsigned i = 0);
```

- 7 *Returns:* `extent_v<T,I>` `std::extent<T,I>::value`, where T is the type represented by `dealias(type)` and I is a constant equal to `i`.

```
constexpr size_t tuple_size(info type);
```

- 8 *Returns:* `tuple_size_v<T>` `std::tuple_size<T>::value`, where T is the type represented by `dealias(type)`.

```
constexpr info tuple_element(size_t index, info type);
```

- 9 *Returns:* A reflection representing the type denoted by `tuple_element_t<I,T>` `std::tuple_element<I,T>::type`, where T is the type represented by `dealias(type)` and I is a constant equal to `index`.

```
constexpr size_t variant_size(info type);
```

- 10 *Returns:* `variant_size_v<T>` `std::variant_size<T>::value`, where T is the type represented by `dealias(type)`.

```
constexpr info variant_alternative(size_t index, info type);
```

- 11 *Returns:* A reflection representing the type denoted by `variant_alternative_t<I,T>` `std::variant_alternative<I,T>::type`, where T is the type represented by `dealias(type)` and I is a constant equal to `index`.

```
constexpr strong_ordering type_order(info t1, info t2);
```

- 12 *Returns:* `type_order_v<T1,T2 std::type_order<T1,T2, where  $T_1$ ,  $T_2$  are the types represented by dealias(t1) and dealias(t2), respectively.`

§ 3 References

[N5014] Thomas Köppe. 2025-08-05. Working Draft, Standard for Programming Language C++. <https://wg21.link/n5014>